

Model Cards & Prompt formats

Llama 3.2

Introduction

This page covers capabilities and guidance specific to the models released with Llama 3.2: The [Llama 3.2 quantized models \(1B/3B\)](#), the [Llama 3.2 lightweight models \(1B/3B\)](#) and the [Llama 3.2 multimodal models \(11B/90B\)](#).

[Download](#) the Llama 3.2 models.

Llama 3.2 Quantized Models (1B/3B)

Introduction

Llama 3.2 included lightweight models in 1B and 3B sizes at bfloat16 (BF16) precision. Subsequent to the release, we updated Llama 3.2 to include *quantized* versions of these models. This section describes these updated lightweight models, how to obtain them, and what use cases they support.

*Note that we have quantized only the **instruct** versions of the Llama 3.2 lightweight models, and that these quantized models have a reduced context length of 8k.*

For in-depth technical information about the Llama 3.2 lightweight models—including the new quantized versions—see the [model card on GitHub](#).

[Download](#) the Llama 3.2 lightweight models.

On this page

Llama 3.2

Introduction

Llama 3.2
Quantized Models
(1B/3B)

Introduction

Fast, Compact,
Accurate—and
Safe

Getting the
Models

Using the
Models

Drop-In
Replacement for
BF16 Models

Quantization
Techniques

Quantization-
Aware Training
and LoRA

SpinQuant

Common
Configuration
Settings

References

Llama 3.2
Lightweight
Models (1B/3B)

Model Card
(1B/3B)

Inference with
lightweight
models

Fast, Compact, Accurate—and Safe

The new quantized models are substantially faster than their non-quantized (BF16) counterparts. The quantized models also have a much lower memory footprint and lower power consumption. At the same time though, they retain nearly the same accuracy as the non-quantized versions.

In addition, because these models were trained and evaluated using Meta’s data and frameworks, they have the same levels of trust and safety as other models in the Llama collection.

The [model card for Llama 3.2](#) has been updated with **performance data** that shows how the quantized models compare with the non-quantized versions.

Getting the Models

You can download the models directly from our [download page](#). Just specify the Llama 3.2 lightweight models (1B/3B) and the quantized versions will be included along with the BF16 versions.

Using the Models

The quantized models are appropriate for any use case that involves constrained memory conditions or the need to conserve power. Typical environments include phones, tablets, and other edge devices, such as smart glasses.

The models have been optimized to use **ExecuTorch** as their runtime environment. The [ExecuTorch repository on GitHub](#) contains a [complete end-to-end example](#) of how to build and deploy the models with ExecuTorch. The example includes guidance to enable you to verify the performance enhancements described above.

The ExecuTorch repository also contains example **demo apps** for [Android](#) and [iOS](#) that you can use to explore potential application use cases.

Drop-In Replacement for BF16 Models

The quantized models are functionally equivalent to the BF16 versions. Prompts designed with the non-quantized models will work without modification on the quantized models. For how to design

Code Interpreter

Tool Calling
(1B/3B)

Function
definitions in
the system
prompt

Function
definitions and
query in the
user prompt

Llama 3.2 Vision
models (11B/90B)

Model Card

Vision Model
Architecture

Vision Model
Inputs and
Outputs

Prompt
Template

Special Tokens

Supported
Roles

Base Model
Prompt

Instruct Model
Prompt

Code Interpreter
and Tool Calling

Similarly, the quantized models are fully compatible with the Llama Guard 3 trust and safety companion models. For more information about leveraging Llama Guard 3 to enhance the safety of your models see the [Llama Guard 3 page](#).

Quantization Techniques

For each model weight, 1B and 3B, we built two quantized versions, for a total of four quantized models. One set of quantized versions uses Quantized Aware Training (QAT) combined with Low-Rank Adaptation (LoRA). The other set uses SpinQuant. This section provides some technical details on these two approaches. For more in-depth information, see the research papers listed in the **References** section below.

Quantization-Aware Training and LoRA

Quantization-Aware Training (QAT) simulates the effects of quantization during the training of the Llama 3.2 models, which enables us to optimize their performance in low precision environments. To initialize QAT, we utilize BF16 Llama 3.2 model checkpoints obtained after supervised fine-tuning (SFT), then perform an additional full round of SFT training with QAT. We then freeze the backbone of the QAT model and perform another round of SFT with low-rank adaptation (LoRA) adaptors applied to all layers within the transformer block. Meanwhile, the LoRA adaptors' weights and activations are maintained in bfloat16, similar to QLoRA.

Finally, we fine-tune the resulting model (both backbone and LoRA adaptors) using direct preference optimization (DPO). The result is a highly efficient model that achieves accuracy that is competitive with the original BF16 model, while maintaining speed and a memory footprint comparable to other quantization methods.

We used [PyTorch Architecture Optimization \(torchao\)](#) to do QAT. You can use QAT as a foundational model and use LoRA to fine-tune Llama for your bespoke use cases, saving time and computational cost.

SpinQuant

SpinQuant is a state-of-the-art technique for post-training quantization. For the SpinQuant models, we utilized [WikiText 2](#), a small calibration dataset, to learn the SpinQuant rotation matrices.

quantization such as range setting and generative post-training quantization (GPTQ). The SpinQuant matrices are optimized for the same quantization scheme as QAT + LoRA.

A key advantage of SpinQuant is its ability to operate without requiring access to training datasets, which are often private. It is an attractive solution for applications where data availability or computational resources are limited.

Some developers might want to quantize their fine-tuned 1B and 3B models, or quantize the models for different targets with different quantization settings. For this reason we also provide the *methodology* for SpinQuant. You can use this methodology to take your own fine-tuned Llama models and quantize them for different hardware targets and use cases with our [open-source SpinQuant repository](#)—which is fully ExecuTorch compatible.

Common Configuration Settings

For both quantization methods, QAT+LoRA and SpinQuant, we used the following quantization scheme:

- We quantize all linear layers in all transformer blocks to a 4-bit groupwise scheme, with a group size of 32, for weights; and 8-bit per token dynamic quantization for activations.
- The classification layer is quantized to 8-bit per channel for weight and 8-bit per token dynamic quantization for activation.
- We employ an 8-bit per channel quantization for embedding.

References

Liu, Zechun, Changsheng Zhao, Igor Fedorov, Bilge Soran, Dhruv Choudhary, Raghuraman Krishnamoorthi, Vikas Chandra, Yuandong Tian, and Tijmen Blankevoort. **SpinQuant: LLM Quantization with Learned Rotations**. arXiv, October 6, 2024.

<https://doi.org/10.48550/arXiv.2405.16406>

Rafailov, Rafael, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. **Direct Preference Optimization: Your Language Model Is Secretly a Reward Model**. arXiv, July 29, 2024. <https://doi.org/10.48550/arXiv.2305.18290>.

Dettmers, Tim, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. **QLoRA: Efficient Finetuning of Quantized LLMs**. arXiv, May 23, 2023. <https://doi.org/10.48550/arXiv.2305.14314>.

Adaptation of Large Language Models. arXiv, October 16, 2021.

<https://doi.org/10.48550/arXiv.2106.09685>.

Llama 3.2 Lightweight Models (1B/3B)

Model Card (1B/3B)

For comprehensive technical information about the Llama 3.2 collection of Lightweight models, please see the official [model card](#), located on GitHub.

[Download](#) the Llama 3.2 lightweight models.

Inference with lightweight models

The recommended way to run inference for these lightweight models on-device is using the [PyTorch ExecuTorch](#) framework. ExecuTorch is an end-to-end solution for enabling on-device inference capabilities across mobile and edge devices including wearables, embedded devices and microcontrollers. It is part of the PyTorch Edge ecosystem and enables efficient deployment of various PyTorch models (vision, speech, Generative AI, and more) to edge devices.

To support our lightweight model launches, ExecuTorch is now supporting bfloat16 with the XNNPack backend in both [Android](#) and [iOS](#), please check out our repository on Github for more technical details as well as end-to-end tutorials.

In addition to the bfloat16 models described above, Llama 3.2 also includes **quantized versions** of the 1B and 3B models. For more information about these quantized versions, see [this section](#).

Prompt Template

The lightweight models share many characteristics with the Llama 3.1 text-only models. For information that is applicable across both sets of models, see the following sections on the Llama 3.1 page.

- [Special Tokens](#)
- [Supported Roles](#)
- [Llama 3.1 Pretrained](#)
- [Llama 3.1 Instruct](#)

Code Interpreter

Tool-calling with the lightweight models can be done in 2 ways:

- Pass the function definitions in the system prompt + pass the query in the user prompt
- Pass the function definitions and query in the user prompt

Note: Unlike the Llama 3.1 larger Models (8B/70B/405B), the lightweight models do not support built-in tools (Brave Search and Wolfram). The lightweight models only support custom functions defined in either the system prompt or user prompt. This decision was made to simplify the user experience of tool-calling with our lightweight models.

Function definitions in the system prompt

Set the function definitions

```
function_definitions = """[
    {
        "name": "get_user_info",
        "description": "Retrieve details for a
specific user by their unique identifier. Note that
the provided function is in Python 3 syntax.",
        "parameters": {
            "type": "dict",
            "required": [
                "user_id"
            ],
            "properties": {
                "user_id": {
                    "type": "integer",
                    "description": "The unique identifier
of the user. It is used to fetch the specific user
details from the database."
                },
                "special": {
                    "type": "string",
                    "description": "Any special
information or parameters that need to be considered
while fetching user details.",
                    "default": "none"
                }
            }
        }
    }
]
```

Set the default system prompt

```
system_prompt = """You are an expert in composing
functions. You are given a question and a set of
possible functions.
Based on the question, you will need to make one or
more function/tool calls to achieve the purpose.
If none of the function can be used, point it out. If
the given question lacks the parameters required by
the function,
also point it out. You should only return the
function call in tools call sections.

If you decide to invoke any of the function(s), you
MUST put it in the format of
[func_name1(params_name1=params_value1,
params_name2=params_value2...), func_name2(params)]\n
You SHOULD NOT include any other text in the
response.

Here is a list of functions in JSON format that you
can
invoke.\n\n{functions}\n""".format(functions=function
_definitions)
```

Set the user query

```
query = "Can you retrieve the details for the user
with the ID 7890, who has black as their special
request?"
```

With the above function definition, system prompt and user query, the input to the LLM looks like:

```
<|start_header_id|>system<|end_header_id|>
You are an expert in composing functions. You are
given a question and a set of possible functions.
Based on the question, you will need to make one or
more function/tool calls to achieve the purpose.
```

~}, the function, also points to user. You should only return the function call in tools call sections. If you decide to invoke any of the function(s), you MUST put it in the format of

```
[func_name1(params_name1=params_value1,
params_name2=params_value2...), func_name2(params)]
```

You SHOULD NOT include any other text in the response.

Here is a list of functions in JSON format that you can invoke. [

```
{
  "name": "get_user_info",
  "description": "Retrieve details for a
specific user by their unique identifier. Note that
the provided function is in Python 3 syntax.",
  "parameters": {
    "type": "dict",
    "required": [
      "user_id"
    ],
    "properties": {
      "user_id": {
        "type": "integer",
        "description": "The unique identifier
of the user. It is used to fetch the specific user
details from the database."
      },
      "special": {
        "type": "string",
        "description": "Any special
information or parameters that need to be considered
while fetching user details.",
        "default": "none"
      }
    }
  }
}
```

```
<|eot_id|><|start_header_id|>user<|end_header_id|>
```

Can you retrieve the details for the user with the ID 7890, who has black as their special request?

```
<|eot_id|>
```

```
<|start_header_id|>assistant<|end_header_id|>
```

And the model responds with the function call that can fulfill the user's query:


```
<|eot_id|>
```

Function definitions and query in the user prompt

You could pass everything in the user prompt as well:

```
<|begin_of_text|>
<|start_header_id|>user<|end_header_id|>
Questions: Can you retrieve the details for the user
with the ID 7890, who has black as their special
request?
Here is a list of functions in JSON format that you
can invoke:
[
  {
    "name": "get_user_info",
    "description": "Retrieve details for a
specific user by their unique identifier. Note that
the provided function is in Python 3 syntax.",
    "parameters": {
      "type": "dict",
      "required": [
        "user_id"
      ],
      "properties": {
        "user_id": {
          "type": "integer",
          "description": "The unique identifier
of the user. It is used to fetch the specific user
details from the database."
        },
        "special": {
          "type": "string",
          "description": "Any special
information or parameters that need to be considered
while fetching user details.",
          "default": "none"
        }
      }
    }
  }
]
Should you decide to return the function call(s), Put
it in the format of [func1(params_name=params_value,
params_name2=params_value2...), func2(params)]
```

With the same response:

```
[get_user_info(user_id=7890, special='black')]  
<|eot_id|>
```

Note that the model's response ends with an `<|eot_id|>` tag indicating end of turn.

Llama 3.2 Vision models (11B/90B)

Note: The Llama 3.2 multimodal models are not accessible from the European Union (EU). Please see the [Llama 3.2 AUP](#) and [Llama FAQ page](#) for more information.

The Llama 3.2 Vision multimodal large language models (LLMs) are a collection of pretrained and instruction-tuned image reasoning generative models in 11B and 90B sizes (text + images in / text out). The Llama 3.2 Vision Instruct models are optimized for visual recognition, image reasoning, captioning, and answering general questions about an image.

Model Card

For comprehensive technical information about the Llama 3.2 Vision models, please see the official [model card](#), located on GitHub.

[Download](#) Llama 3.2 Vision models.

Vision Model Architecture

The Llama vision models are a late-fusion architecture with cross-attention layers that process text tokens and image tokens (from the vision encoder) efficiently. To read more about the architecture refer to page 56 of the [Llama 3 paper](#).

Vision Model Inputs and Outputs

output of the model is text only.

With **text-only** inputs, the Llama 3.2 Vision models are functionally the same as the [Llama 3.1 Text models](#); this allows the Llama 3.2 Vision models to be a drop-in replacement for Llama 3.1 8B/70B with added image-understanding capabilities.

Prompt Template

Special Tokens

The vision model supports all the [tokens available in the text-only models](#), along with a new special token `<|image|>` which represents the passed image.

Supported Roles

There are 4 different roles that are supported by Llama text models:

1. **system**: Sets the context in which to interact with the AI model. It typically includes rules, guidelines, or necessary information that help the model respond effectively.
2. **user**: Represents the human interacting with the model. It includes the inputs, commands, and questions to the model.
3. **ipython**: A new role introduced in Llama 3.1. Semantically, this role means "tool". This role is used to mark messages with the output of a tool call when sent back to the model from the executor.
4. **assistant**: Represents the response generated by the AI model based on the context provided in the `system`, `ipython` and `user` prompts.

```
[system, assistant, user, ipython]
```

Base Model Prompt

The prompt to the base vision model consists of the `<|image|>` tag along with the text to continue generating

```
<|begin_of_text|><|image|>If I had to write a haiku  
for this one
```

The prompt to the vision instruct model is similar to the text instruct model, with the additional `<|image|>` tag if the input includes an image to reason about.

```
<|begin_of_text|>
<|start_header_id|>user<|end_header_id|>

<|image|>Describe this image in two
sentences<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

Two things to **note** in the instruct model prompt:

- We don't need a **system** prompt when passing an image to the model; the **user** prompt will contain the `<|image|>` tag and text query.
- The position of the `<|image|>` tag is important! The image immediately preceding a query is used to answer the query, **make sure the text query follows the `<|image|>` tag**. This is controlled by the cross-attention layer mask in the model.

For more examples of the vision prompt template, please refer to [vision_prompt_format.md](#) in the [meta-llama](#) GitHub repository.

Code Interpreter and Tool Calling

With **text-only** inputs, the [code interpreter](#) and [tool-calling](#) capabilities of the Llama 3.2 Vision Models work exactly like their [Llama 3.1 Text Model counterparts](#). You can use either the **system** or **user** prompts to provide the function definitions.

Currently the vision models don't support tool-calling with **text+image** inputs.

Was this page helpful?



Products

- API
- API waitlist
- Login

Models

- Llama 4
- Llama 3
- Download

News

- AI at Meta Blog
- Meta Newsroom

Documentation

- Overview
- Models
- How to guides
- Deployment guides
- Integration guides

Learn

- Cookbook
- Videos
- FAQ

AI research

- Overview

Community

- Case studies
- AI research community
- Llama Impact Grants
- Hugging Face
- GitHub

Llama Protections

- Overview
- Llama Defenders Program
- Developer Use Guide

Terms & policies

- Terms of Service
- Privacy Policy
- Cookies